

Advanced C++20 Coroutines: Mastering `co_await` and Task

Implementing custom awaitables and task types for real-world asynchronous programming patterns.

15 min read · Mar 26, 2026

[Open in app](#) ↗

≡ Medium

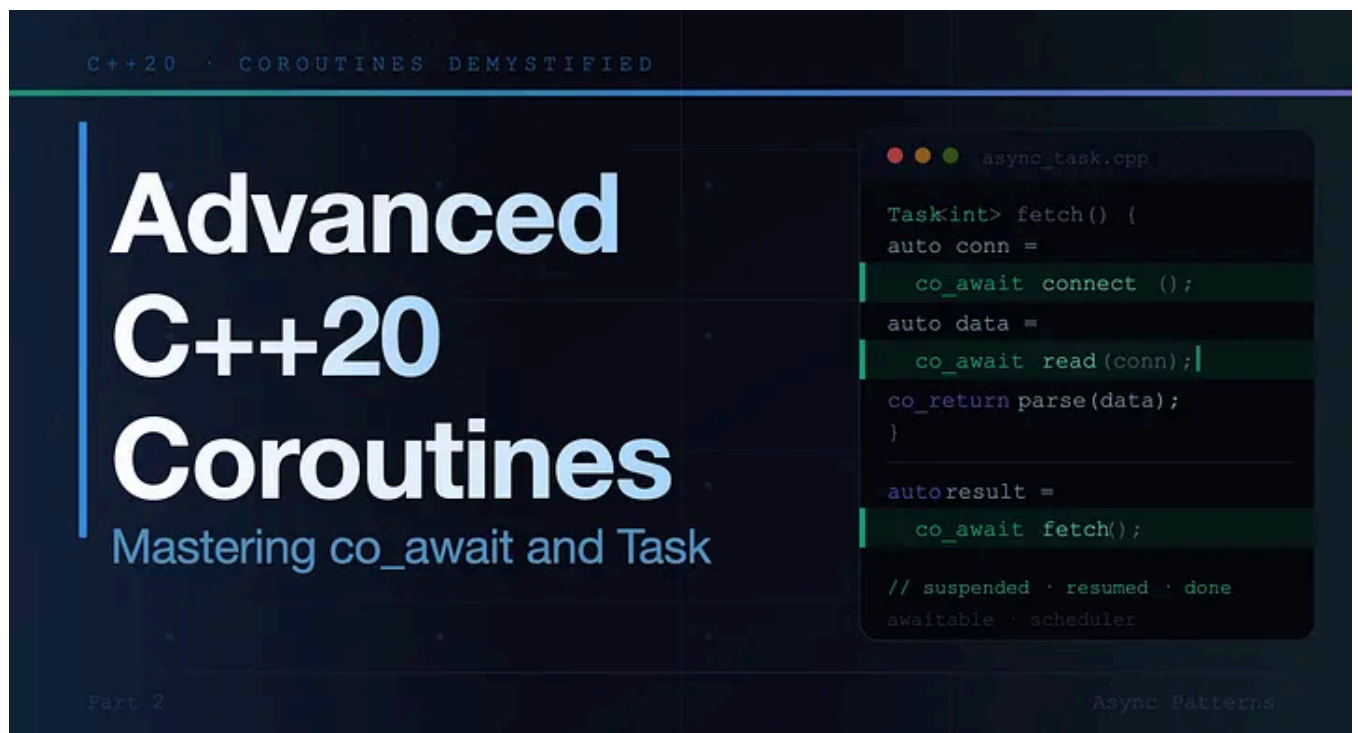


A

🔊 Listen

📄 Share

⋮ More



Welcome back! In [Part 1\(C++20 Coroutines Demystified: Build a Generator From Scratch\)](#), we built a solid mental model and a working `Generator<T>`. Now it's time to peel back the remaining layers. We'll tackle the real primitive behind everything

(`co_await`), build custom awaitables, create a `Task<T>` type, and cover the patterns and pitfalls that separate tutorial code from production code.

Understanding `co_await`: The Real Primitive

In Part 1 I told you `co_yield` is actually just syntactic sugar. Let's prove it:

```
+-----+
| What you write:      What the compiler actually generates: |
| =====            ===== |
| co_yield 42;         --> co_await promise.yield_value(42);   |
| co_return 42;        --> promise.return_value(42);          |
|                      goto final_suspend;                    |
| co_return;           --> promise.return_void();              |
|                      goto final_suspend;                    |
| co_await expr;       --> (the 3-step awaitable dance -- below) |
+-----+
```

So `co_await` is the real primitive. Everything boils down to it. But what does `co_await` actually do?

Roughly speaking, `co_await a;` does the following three things:

1. **Saves local state to the heap.** Ensures all local variables in the current coroutine are saved to the heap-allocated coroutine frame, so they survive the suspension.
2. **Creates a resume callable.** Creates a callable object (the `coroutine_handle`) that, when invoked, will resume execution of the coroutine at the point immediately following the `co_await` expression.
3. **Hands control to the awaitable.** Calls a method on the target object `a`, passing it that resume handle. It is then `a`'s responsibility to decide when — or whether — to resume the coroutine.

This is the heart of the whole system. The awaitable object receives the handle and can store it, schedule it on a thread pool, attach it to an I/O completion callback, or resume it immediately. The coroutine doesn't care *how* it gets resumed — it only cares *that* it eventually will be.



The three methods (`await_ready` , `await_suspend` , `await_resume`) are how the awaitable communicates each of those decisions back to the compiler-generated machinery.

The Three-Step Awaitable Dance

When the compiler sees `co_await some_expression` , it evaluates the expression to get an **awaitable** object, and then calls three methods on it:

```
+-----+
|
|  co_await some_awaitable;
|
|  +-----+
|  | Step 1: await_ready()
|  |
|  | "Hey awaitable, is the result already available?"
|  |
|  | Returns true --> Skip suspension! Continue immediately.
|  | Returns false --> We need to suspend. Go to Step 2.
|  |
|  | +-----+
|  | | (returned false)
|  | v
|  | +-----+
|  | | Step 2: await_suspend(coroutine_handle h)
|  | |
|  | | "Okay, we're suspending. Here's the handle to resume
|  | | this coroutine later. Do whatever you need."
|  |
|  |
```

```

| | Can return:
| | void          --> always suspend
| | bool          --> true=suspend, false=don't suspend
| | coroutine_handle --> suspend me, resume THAT coroutine
| |                  (symmetric transfer)
|
| COROUTINE IS NOW SUSPENDED
| (control returns to whoever called resume() or to
| the initial caller)
|
+-----+-----+
| | | (someone calls handle.resume() later)
| | |
| | v
|
+-----+-----+
| | Step 3: await_resume()
|
| "Welcome back! Here's the result of the co_await
| expression."
|
| This is the VALUE that co_await returns.
| Can be void, int, string, whatever.
|
+-----+-----+
+-----+-----+

```

The standard library gives you two pre-built awaitables:

std::suspend_always =====	std::suspend_never =====
await_ready() --> false	await_ready() --> true
await_suspend() --> (void)	await_suspend() --> (void)
await_resume() --> (void)	await_resume() --> (void)
"Always pause"	"Never pause, keep going"

If you remember one thing:

await_suspend decides who resumes next.

How `co_yield` Flows Through `co_await` ?

Now you can see the full picture of what happens with `co_yield`:

```
co_yield 42;
|
v
co_await promise.yield_value(42);
|
| yield_value() stores 42 in current_value,
| then returns suspend_always{}
|
v
co_await suspend_always{};
|
+-- await_ready()    --> false  (always suspend)
|
+-- await_suspend(h) --> void   (just suspend, nothing fancy)
|
+-- SUSPENDED with value 42 stored in promise
    .
    . (caller reads value, then calls handle.resume())
    .
    await_resume() --> void   (no return value for co_yield)
    execution continues after the co_yield statement
```

Let's Build a Custom Awaitable

Now that you understand the three methods, let's make something — an awaitable that transforms a value:

```
#include <coroutine>
#include <iostream>
#include <string>

// A custom awaitable that adds excitement to strings
struct AddExcitement {
    std::string input;

    bool await_ready() const noexcept {
        std::cout << " [await_ready: not yet!]\n";
        return false;
    }
};
```

```

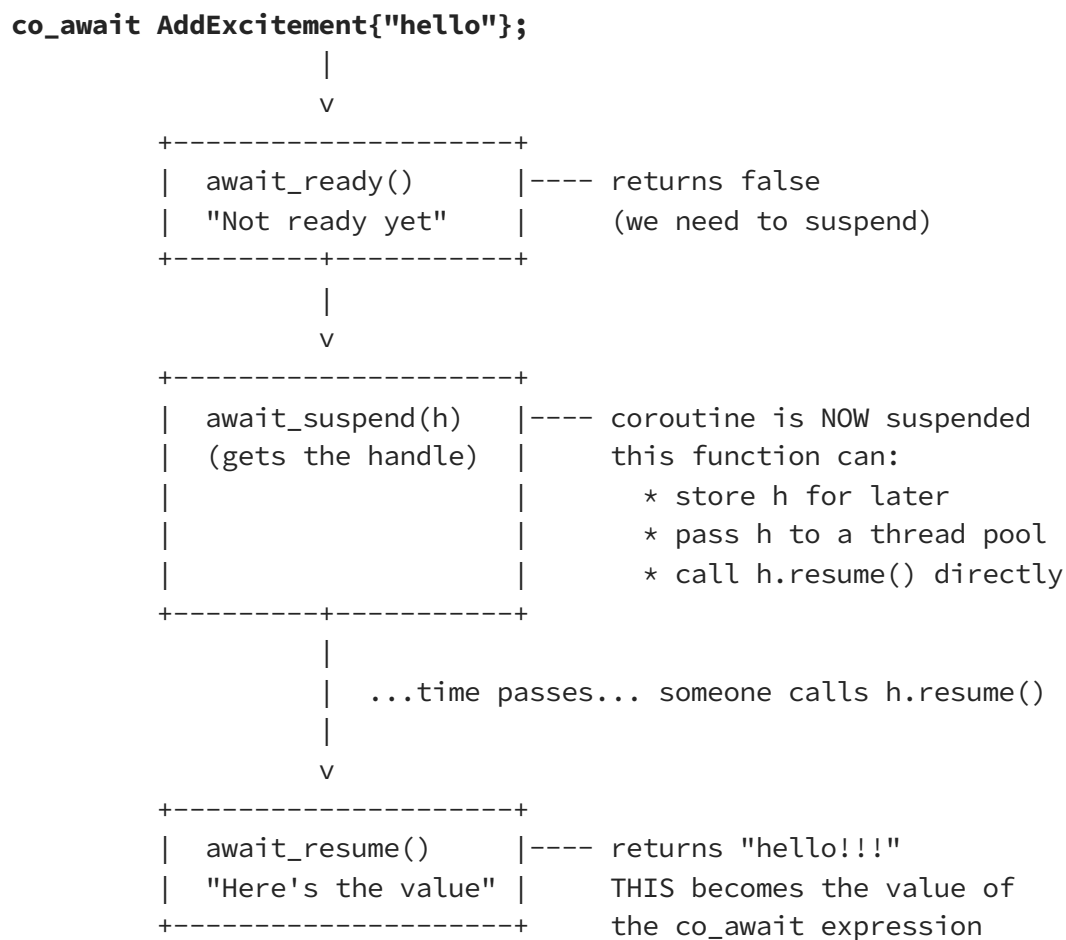
}

// Returning void = always truly suspend.
// In a real system, you'd stash the handle for some I/O callback
// or thread pool to call later. Here we resume immediately for demo.
void await_suspend(std::coroutine_handle<> h) const noexcept {
    std::cout << "  [await_suspend: pausing coroutine...]\n";
    h.resume(); // In real life: schedule h.resume() on an event loop
}

// The value that the co_await expression produces
std::string await_resume() const noexcept {
    std::cout << "  [await_resume: here's your result!]\n";
    return input + "!!!";
}
};

```

Here's the flow visualized:



Cool... but what does this look like in real life?

A Real Async Example: Simulating `co_await sleep()`

```
#include <coroutine>
#include <iostream>
#include <thread>
#include <chrono>

// =====
//  A minimal "fire-and-forget" task
// =====
struct FireAndForget {
    struct promise_type {
        FireAndForget get_return_object() { return {}; }

        std::suspend_never initial_suspend() noexcept { return {}; }
        std::suspend_never final_suspend() noexcept { return {}; }

        void return_void() {}
        void unhandled_exception() { std::terminate(); }
    };
};

// =====
//  A fake async "sleep" awaitable
// =====
struct Sleep {
    std::chrono::milliseconds duration;

    // Always suspend
    bool await_ready() const noexcept { return false; }

    // Called when coroutine suspends
    void await_suspend(std::coroutine_handle<> handle) const {
        std::thread([handle, d = duration]() {
            std::this_thread::sleep_for(d);
            std::cout << " [timer done, resuming coroutine]\n";
            handle.resume(); // Resume after delay
        }).detach();
    }

    void await_resume() const noexcept {}
};

// =====
//  A coroutine using async sleep
// =====
FireAndForget async_example() {
```

```

std::cout << "Start\n";

co_await Sleep{std::chrono::milliseconds(1000)};

std::cout << "After 1 second\n";

co_await Sleep{std::chrono::milliseconds(500)};

std::cout << "After another 0.5 seconds\n";
}

// =====
//  main()
// =====
int main() {
    async_example();

    std::cout << "Main continues immediately!\n";

    // Prevent program from exiting too early
    std::this_thread::sleep_for(std::chrono::seconds(2));
}

```

Key snippet:

```
co_await Sleep{1000ms};
```

Does **NOT** block the thread. Instead:

1. Coroutine hits `co_await`
2. `await_suspend(handle)` runs
3. A background thread is started
4. Coroutine is **suspended**
5. `main()` keeps running
6. After 1 second → `handle.resume()`
7. Coroutine continues exactly where it left off

```

async_example() starts
|
v
print "Start"
|
v
co_await Sleep(1s)
|
+--> await_suspend(handle)
|
+--> spawn thread
|
+--> coroutine SUSPENDED
|
v
main() continues immediately
|
v
(after 1 second)
|
+--> thread calls handle.resume()
|
v
print "After 1 second"

```

Note: This is a simplified demo using `std::thread`.

Real async runtimes (Boost.Asio, cppcoro, libunifex):

- Use event loops instead of spawning threads
- Avoid creating one thread per await
- Integrate with I/O systems (epoll, IOCP, etc.)

*This is the moment everything clicks: `co_await` doesn't 'wait' — it **hands control to something else and trusts it to resume later**.*

A Simple Task Type: Using `co_await` and `co_return`

Generators are cool, but let's build a `Task` — a coroutine that produces a single result. This is the foundation for async programming:

```

#include <coroutine>
#include <iostream>
#include <exception>
#include <optional>

template <typename T>
class Task {
public:
    struct promise_type {
        std::optional<T> result;
        std::exception_ptr exception;

        Task get_return_object() {
            return Task{
                std::coroutine_handle<promise_type>::from_promise(*this)
            };
        }

        // Lazy: don't start until explicitly resumed
        std::suspend_always initial_suspend() noexcept { return {}; }

        // Stay alive so we can read the result
        std::suspend_always final_suspend() noexcept { return {}; }

        // co_return value; stores the result
        void return_value(T value) {
            result = std::move(value);
        }

        void unhandled_exception() {
            exception = std::current_exception();
        }
    };

    explicit Task(std::coroutine_handle<promise_type> h) : handle_(h) {}

    ~Task() {
        if (handle_) handle_.destroy();
    }

    Task(const Task&) = delete;
    Task& operator=(const Task&) = delete;
    Task(Task&& o) noexcept
        : handle_(std::exchange(o.handle_, nullptr)) {}
    Task& operator=(Task&& o) noexcept {
        if (this != &o) {
            if (handle_) handle_.destroy();
            handle_ = std::exchange(o.handle_, nullptr);
        }
        return *this;
    }
}

```

```

// Run the coroutine to completion and get the result
T get() {
    handle_.resume();
    auto& promise = handle_.promise();
    if (promise.exception) {
        std::rethrow_exception(promise.exception);
    }
    return std::move(*promise.result);
}

private:
    std::coroutine_handle<promise_type> handle_;
};

```

Usage:

```

Task<int> compute_answer() {
    int result = 42;
    co_return result;
}

Task<std::string> greet(std::string name) {
    co_return "Hello, " + name + "!";
}

int main() {
    auto task1 = compute_answer();
    std::cout << task1.get() << "\n"; // 42

    auto task2 = greet("World");
    std::cout << task2.get() << "\n"; // Hello, World!
}

```

Task Lifecycle:

=====

```

auto t = compute_answer();    <-- Frame allocated, initial_suspend
                               (body NOT running yet)

    |
    v
t.get()                        <-- handle_.resume() kicks off body
    |
    | +-----+
    | | Body runs... |
    | | result = 42  |
    | |              |

```

```

    | | co_return result; |
    | |   -> return_value(42) |
    | |   -> final_suspend SUSPENDED |
    | +-----+
    |
    v
promise.result -> 42
return to caller

```

This is a **minimal, synchronous Task** for learning. Real async tasks typically:

- resume on executors
- support continuations
- integrate with event loops

Making Generator Work with Range-Based For Loops

Our generator from Part 1 works, but calling `.next()` and `.value()` is verbose. Let's make it work with `for (auto x : gen)` by adding iterator support:

```

template <typename T>
class Generator {
public:
    struct promise_type { /* ... same as Part 1 ... */ };

    // =====
    //  ITERATOR SUPPORT
    // =====

    struct sentinel {}; // Empty tag type -- marks "end"

    struct iterator {
        std::coroutine_handle<promise_type> handle;

        iterator& operator++() {
            handle.resume(); // Advance to next co_yield
            return *this;
        }

        T operator*() const {
            return handle.promise().current_value;
        }
    };
};

```

```

    }

    bool operator==(sentinel) const {
        return handle.done(); // Are we past the last co_yield?
    }
};

iterator begin() {
    handle.resume(); // Get the FIRST value
    return {handle_};
}

sentinel end() {
    return {};
}

// ... constructor, destructor, move ops same as Part 1 ...

private:
    std::coroutine_handle<promise_type> handle_;
};

```

Now you can write:

```

Generator<int> fibonacci() {
    int a = 0, b = 1;
    while (true) { // Infinite sequence! No problem!
        co_yield a;
        auto next = a + b;
        a = b;
        b = next;
    }
}

int main() {
    int count = 0;
    for (int fib : fibonacci()) {
        std::cout << fib << " ";
        if (++count >= 10) break; // Take only what you need
    }
    // Output: 0 1 1 2 3 5 8 13 21 34
}

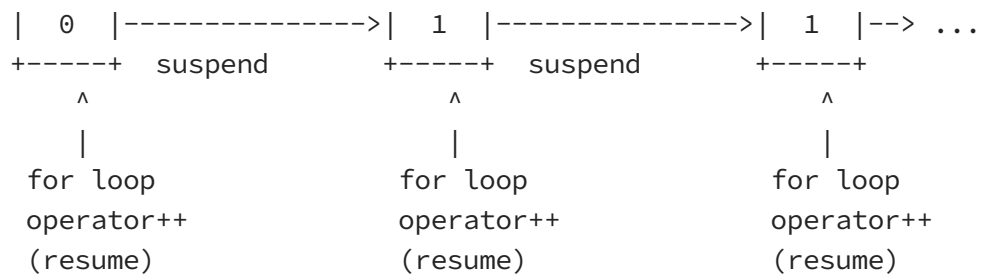
```

fibonacci() produces values LAZILY:

```

+-----+ co_yield 0    +-----+ co_yield 1    +-----+

```



The infinite `while(true)` NEVER runs away because the coroutine only advances when asked!

Coroutine Composition: Generators Calling Generators

Here's where things get really nice — coroutines can compose like Unix pipes:

```

// A generator that yields lines from a "file" (simulated)
Generator<std::string> read_lines() {
    co_yield "INFO: Server started";
    co_yield "DEBUG: Loading config";
    co_yield "ERROR: Disk full!";
    co_yield "INFO: Request received";
    co_yield "ERROR: Connection refused";
    co_yield "DEBUG: Cache miss";
}

// A generator that FILTERS another generator
Generator<std::string> only_errors(Generator<std::string> source) {
    for (auto line : source) {
        if (line.starts_with("ERROR")) {
            co_yield line; // Only pass through errors
        }
        // Non-errors are silently skipped -- lazy!
    }
}

int main() {
    // Compose generators like: read_lines() | grep "ERROR"
    auto errors = only_errors(read_lines());

    for (const auto& err : errors) {
        std::cout << err << "\n";
    }
    // Output:
    // ERROR: Disk full!
  
```

```
// ERROR: Connection refused
}
```

read_lines()	only_errors()	main()
<-- resume -----	<-- resume -----	for loop
yield "INFO: ..." ----->	starts_with("ERROR")? NO	
	skip, resume source	
<-- resume -----		
yield "DEBUG: ..." ----->	starts_with("ERROR")? NO	
	skip, resume source	
<-- resume -----		
yield "ERROR: Disk" ----->	YES! co_yield it! ----->	print
	SUSPENDED	
	<-- resume -----	next iter
<-- resume -----		
yield "INFO: ..." ----->	NO, skip	
<-- resume -----		
yield "ERROR: Conn" ----->	YES! co_yield! ----->	print
	SUSPENDED	
...	...	

Values flow LAZILY through the chain!
Nothing is buffered. Memory usage: 0(1).

Symmetric Transfer: The Cool Kid's `await_suspend`

When `await_suspend` returns a `coroutine_handle`, the compiler will suspend the current coroutine and immediately resume the returned one without adding a stack frame. This is called **symmetric transfer**, and it prevents stack overflow when coroutines ping-pong between each other:

WITHOUT Symmetric Transfer

=====

```
resume(A) -> A runs -> resume(B) -> B runs -> resume(A) -> ...
      |           |           |
      v           v           v
```

+-----+	+-----+	+-----+	
Stack	Stack	Stack	
frame A	frame A	frame A	Stack grows
	frame B	frame B	and grows...
		frame A	STACK OVERFLOW!
+-----+	+-----+	+-----+	

WITH Symmetric Transfer

=====

```
resume(A) -> A suspends, returns handle to B
            -> B resumes (REPLACES A on stack!)
            -> B suspends, returns handle to A
            -> A resumes (REPLACES B!)
```

+-----+	+-----+	+-----+	
Stack	Stack	Stack	Stack stays
frame A	frame B	frame A	FLAT!
+-----+	+-----+	+-----+	

You achieve this by having `await_suspend` return a handle:

```
struct my_awaitable {
    std::coroutine_handle<> other;

    bool await_ready() { return false; }

    // Returns a handle --> symmetric transfer!
    std::coroutine_handle<> await_suspend(
        std::coroutine_handle<> current
    ) noexcept {
        // Suspend current, resume other -- no stack buildup
        return other;
    }

    void await_resume() {}
};
```

Here's the full `await_suspend` return type reference:

+-----+	+-----+
void	Always suspend. Control returns to the

	caller of resume() / initial caller.
bool	true --> suspend (like void) false --> don't suspend, keep running (useful for "suspend only if needed")
coroutine_ handle<>	Suspend THIS coroutine, immediately resume the RETURNED coroutine. (Symmetric transfer -- stack stays flat)
	Return noop_coroutine() to suspend without resuming anyone.
+-----+-----+	

Exception Propagation: Wormholes for Errors

In production code, you want exceptions from inside the coroutine to propagate to the caller. Here's the pattern:

```
struct promise_type {
    std::exception_ptr eptr; // Store the exception

    void unhandled_exception() {
        eptr = std::current_exception(); // Capture it!
    }
    // ... rest of promise ...
};
// In your return type's "get result" method:
T get() {
    handle_.resume();
    if (handle_.promise().eptr) {
        std::rethrow_exception(handle_.promise().eptr);
        // Exception teleports from coroutine body to caller!
    }
    return std::move(*handle_.promise().result);
}
```

Coroutine body

```
|
| throw runtime_error("oops");
|
```

Caller

```
|
|
|
```

```

      v
unhandled_exception()
eptr = current_exception()
exception saved!
      |
final_suspend() -> SUSPENDED
      |
      task.get()
      |
      handle_.resume()... done
      eptr != null? YES!
      rethrow_exception(eptr)
      |
      v
      catch (runtime_error& e)
      "oops"

```

The exception SURVIVES across the suspend/resume boundary!
It's stored in the promise and rethrown when the caller asks.

Things to watchout

1: Dangling references to parameters

BAD:

```
Generator<char> bad(const std::string& s) { // <- reference!
    for (char c : s) co_yield c;
}
auto gen = bad(std::string("hello"));
// The temporary string is DEAD. gen holds a dangling ref.
```

GOOD :

```
Generator<char> good(std::string s) { // <- by value!
    for (char c : s) co_yield c;
}
// String is COPIED into the coroutine frame. Safe!
```

2: Using `suspend_never` for `final_suspend`

If `final_suspend` returns `suspend_never`:

- * Frame is destroyed immediately when the coroutine body ends
- * Your coroutine_handle is now DANGLING
- * Calling handle_.done() or handle_.destroy() --> UB

Rule: ALWAYS use `suspend_always` for `final_suspend`.
Let your return type's destructor call `handle_.destroy()`.

3: Forgetting the coroutine is lazy

```
Generator<int> gen = count_up_to(5);  
// NOTHING has executed yet! Not a single line of the body!  
// You must call .next() or .resume() to start it.
```

4: Having BOTH `return_void()` AND `return_value()`

Pick one. Never both. The compiler will be very upset.

5: Resuming a done coroutine

```
if (handle_.done()) {  
    handle_.resume(); // UNDEFINED BEHAVIOR  
}  
Always check .done() before calling .resume()!
```

The Grand Unified Diagram

The complete flow of a C++20 coroutine from birth to death:

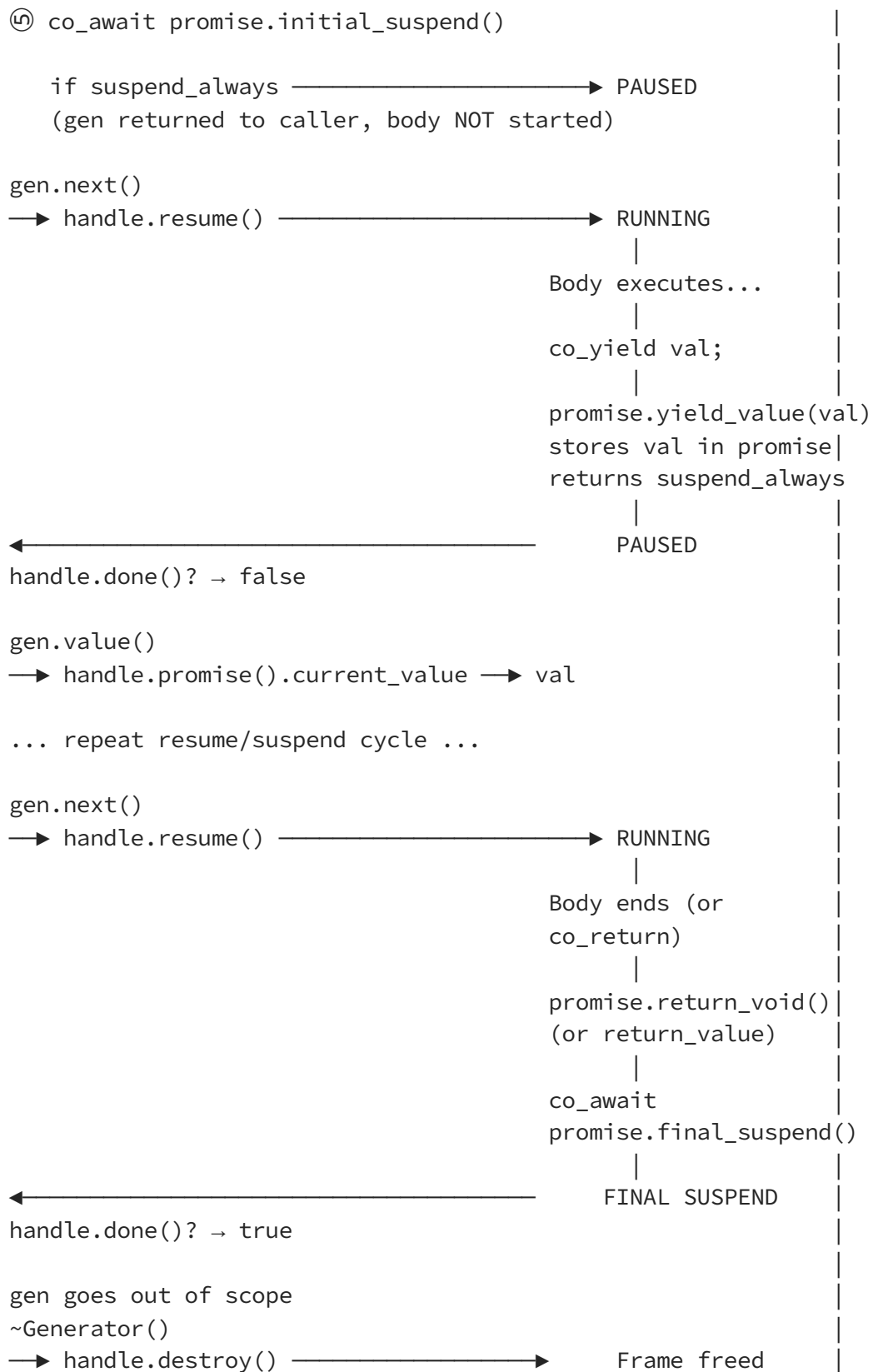
C++20 COROUTINE: THE COMPLETE PICTURE

CALLER (main, event loop, etc.)

COROUTINE

```
auto gen = my_coroutine(args);
```

- ① Compiler allocates frame
- ② Copies args into frame
- ③ Constructs `promise_type` in frame
- ④ Calls `promise.get_return_object()` ← (gen now holds a `coroutine_handle`)



Whenever you need a new coroutine return type, use this checklist:

COROUTINE RETURN TYPE CHECKLIST

```
struct MyCoroutine {
```

```
    struct promise_type {
```

REQUIRED

=====

```
        [ ] get_return_object()    Always needed
```

```
        [ ] initial_suspend()      Always needed (return awaitable)
```

```
        [ ] final_suspend() noexcept Always needed (return awaitable)
```

```
        [ ] unhandled_exception()   Always needed
```

```
        Pick ONE:
```

PICK ONE

```
        [ ] return_void()           =====
```

```
        [ ] return_value(T)         (depends on co_return usage)
```

```
        Optional:
```

OPTIONAL

```
        [ ] yield_value(T)           =====
```

(only if using co_yield)

```
        [ ] await_transform(expr)    (intercept co_await expressions)
```

```
    };
```

```
    [ ] coroutine_handle<promise_type> member
```

```
    [ ] Constructor from handle
```

```
    [ ] Destructor calls handle.destroy()
```

```
    [ ] Delete copy, enable move
```

```
    [ ] Public API for your use case (next, value, get, etc.)
```

```
};
```

The One-Page Summary

C++20 COROUTINES IN 60 SECONDS

=====

1. A coroutine is a function that can SUSPEND and RESUME
2. It becomes a coroutine by using `co_await`, `co_yield`, or `co_return`
3. You must provide:
 - * A return type (e.g., `Generator<T>`, `Task<T>`)
 - * A nested `promise_type` inside that return type
 - * The `promise_type` has hooks the compiler calls automatically
4. The compiler handles:
 - * Heap allocation of the coroutine frame
 - * Saving/restoring state across suspensions
 - * Calling your `promise_type` methods at the right times
5. `coroutine_handle<promise_type>` is your remote control:
 - * `.resume()` --> continue running
 - * `.done()` --> check if finished
 - * `.destroy()` --> free the frame
 - * `.promise()` --> access the promise object
6. An "awaitable" has three methods:
 - * `await_ready()` --> can we skip suspension?
 - * `await_suspend()` --> what to do when suspending
 - * `await_resume()` --> what value does `co_await` produce
7. Common types:
 - * **Generator**: uses `co_yield`, lazy, produces sequences
 - * **Task**: uses `co_return`, produces a single result
8. Always use `suspend_always` for `final_suspend()`. Trust me.
9. Take parameters BY VALUE to avoid dangling references.
10. The standard gives you the engine. YOU build the car.
(Or use a library like `cppcoro`, `libunifex`, etc.)

Where To Go From Here

You now understand the complete machinery of C++20 coroutines. Here's what to explore next: ▶

- **cppcoro library** — Pre-built `Task`, `Generator`, `async_generator`, and more

- **libunifex** — Facebook’s async framework built on coroutines
- **Boost.Asio** — Excellent coroutine integration for networking/I/O
- **await_transform** — A `promise_type` hook that intercepts every `co_await` in your coroutine body (great for cancellation, logging, type restrictions)
- **Coroutine allocator customization** — Override `operator new / operator delete` in the `promise_type` to control where the frame is allocated (stack memory, memory pools, arenas)

Final Words

C++20 coroutines are like IKEA furniture. The end result is beautiful and functional. The assembly process involves strange tools, cryptic instructions, and at least one moment where you question every life decision that led you here.

But once you’ve built your `Generator<T>` and your `Task<T>`, you'll never go back. Lazy sequences, async I/O, state machines — they all become elegant.

Now go write some coroutines. And remember: when the compiler yells at you about a missing `promise_type`, take a deep breath, look at the checklist, and fill in the blanks.

You’ve got this.

C Plus Plus Language

Cpp20

Software Development

Asynchronous Programming

Programming



Follow

Published in Towards Dev

9.8K followers · Last published 19 hours ago